

Relatório de Auditoria de Segurança — news_portal

Komuniki (komuniki.com.br) e Blog da Kelly (kellyfarias.com.br/news) — Django + Wagtail

Data: 25/09/2026 · Commit auditado: 65f486d (master, merge do PR #54)

0 CRÍTICA 3 ALTA 3 MÉDIA 3 BAIXA 3 INFORMATIVA 34 PONTOS FORTES

Escopo auditado

Repositório	Sitr3n01/news_portal — commit 65f486d (master, merge do PR #54)
Stack detectada	Python 3.12 · Django 5.x (5.2.17 no ambiente de verificação) · Wagtail 7.4 · Django Unfold 0.87 · PostgreSQL 16 · templates Django + HTMX + Alpine.js · Docker Compose + Nginx + Cloudflare · GitHub Actions
Autenticação	Sessão do Django (backend em banco) + django-axes; login unificado (/entrar/); OpenID Connect do Google opcional; papéis = campo role → grupos/permisões de modelo (apps/accounts/admin_roles.py)
Escopo auditado	apps/ (10 apps, todos os handlers de rota), config/, templates/, static/ (código próprio; vendor/ só na varredura de chaves), docker/, scripts/deploy/, .github/workflows/, docs/ e o histórico git completo (182 commits)
Fora do escopo	Servidor de produção (sem acesso), CVEs de dependências de terceiros, testes de carga e engenharia social
Verificação	Leitura estática linha a linha + 8 testes-sonda dinâmicos (pytest + Django test client, em venv isolado, descartados após a execução) + execução de manage.py check --deploy com os placeholders do repositório

Nota metodológica — como cada categoria foi mapeada para a stack

Categoria	Equivalente na stack detectada	Como foi verificado
1. Banco sem tranca	Django + PostgreSQL, sem RLS. O isolamento é feito na aplicação: Sites framework (FK site + CurrentSiteManager "on_site", SITE_ID=1 fixo), recorte por dono (request.user) e permissões de modelo por cargo.	Leitura de todas as queries das views públicas, do painel e do admin, conferindo on_site, status e filtro por usuário.
2. Permissão no navegador	Não há SPA: o "frontend" são templates Django, Unfold e Wagtail. Gates de interface: navegação do painel (navigation.py), fieldsets do admin, rótulos "só superusuário". No servidor: ModelAdmin.has_*_permission, permission policies do Wagtail e decorators.	Cruzamento de cada NavItem/gate de papel com a view ou ModelAdmin de destino; sondas dinâmicas com usuários de cada cargo.
3. IDOR	Rotas com <int:...>, <slug:...> e <path:token> em apps/*/urls.py, change views do Django admin e viewsets do Wagtail.	Inventário completo dos handlers (seção 2.3) e checagem de posse/site/status em cada get_object.
4. Chaves expostas	Settings via django-enviro (env/.env.prod), Docker Compose, GitHub Actions, scripts de deploy e docs. O "bundle" do frontend é static/ servido por WhiteNoise/nginx (sem build JS).	Grep de padrões de segredo na árvore, git log -p em 182 commits (clone não-raso), static/ e templates; execução do check de deploy com os placeholders.
5. XSS	Templates Django com autoescape + HTMX + Alpine.js; HTML de usuário via bleach (sanitize_html) e StreamField do Wagtail; uploads servidos pelo nginx em /media/; e-mails via render_to_string.	Busca de sinks (l_safe, mark_safe, format_html, autoescape off, x-html, innerHTML, eval); revisão dos blocos, da sanitização, da CSP e dos headers do nginx; upload dinâmico de HTML/SVG.

1. Resumo executivo

12 Achados	0 Crítica	3 Alta	3 Média	3 Baixa	3 Informativa	34 Pontos fortes
----------------------	---------------------	------------------	-------------------	-------------------	-------------------------	----------------------------

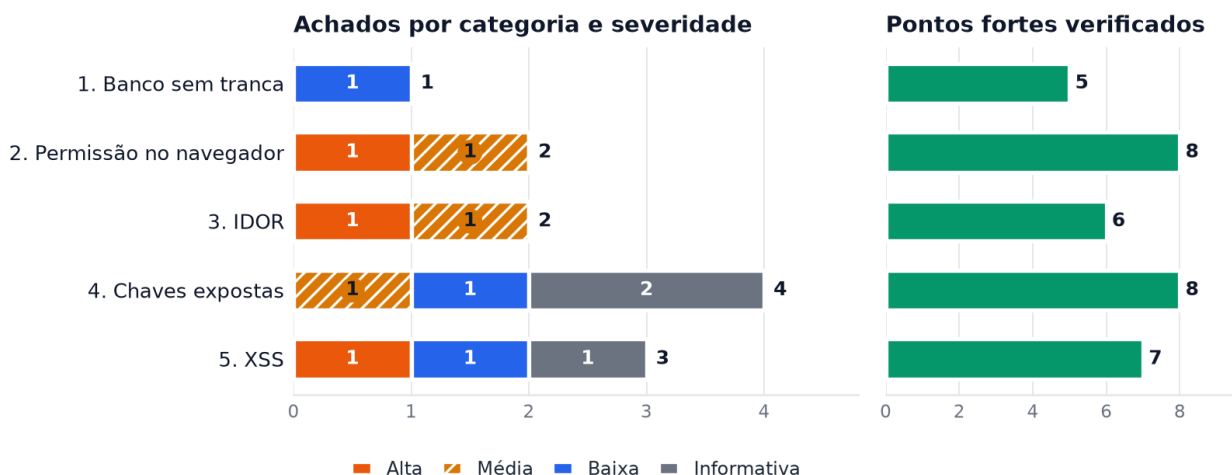
Foram percorridos todos os handlers de rota do projeto (inventário na seção 2.3), a configuração de deploy e os 182 commits do histórico. Resultado: **12 achados verificados** — nenhum crítico, 3 altos, 3 médios, 3 baixos e 3 informativos — e **34 controles confirmados como corretos**. Os três achados altos têm a mesma raiz: uma regra que a interface respeita, mas que o servidor não aplica em todos os caminhos — o limite de superusuário (PRIV-01), o filtro de publicação nas rotas por ID (IDOR-01) e o tipo de arquivo aceito em uploads servidos na mesma origem dos painéis (XSS-01). Encadeados, XSS-01 e PRIV-01 levam uma conta de conteúdo até superusuário mesmo quando quem abre o link é só um Administrador Geral. Todos os achados com exploração foram confirmados em teste dinâmico.



Leitura rápida

- Nenhum achado crítico; os 3 altos tiveram a exploração confirmada em teste.
- Nenhum segredo real no código, no bundle ou nos 182 commits: o risco de chaves é de processo (SEC-01).
- Sanitização de HTML, autoescape e JS próprio sem sinks: não há XSS por texto; o vetor é upload (XSS-01).
- As correções P1 são pequenas e locais: admin de usuários, duas views e allowlist + nginx.

Paleta fixa do relatório: alta e média são tons próximos; por isso os valores vêm escritos em cada marca e a média leva hachura.



Escala de severidade. **Crítica:** exploração remota sem autenticação com comprometimento amplo. **Alta:** escalada de privilégio ou vazamento relevante com barreira baixa. **Média:** exige papel interno ou uma configuração específica. **Baixa:** condição improvável ou só defesa em profundidade. **Informativa:** sem exploração hoje; higiene. Só entram achados verificados no código real: cada um traz arquivo:linha, trecho e como foi confirmado.

2. Pontos fortes e pontos fracos

2.1 Pontos fortes — o que está protegido

Cada item foi conferido no código do commit auditado; a evidência aponta onde a proteção mora.

Categoria	Proteção verificada	Evidência (arquivo:linha)
1. Banco sem tranca	Listagens, detalhe, busca, feeds e sitemap públicos filtram por Site e por publicação	apps/news/views.py:83, 119, 152, 186, 206, 226, 255-266, 297, 326 · apps/news/feeds.py:19-20, 57-58 · apps/news/sitemaps.py:21-22
1. Banco sem tranca	Portal da escola isolado por Site e publicação	apps/school/views.py:179, 185, 203 (on_site + site=request.site + is_published)
1. Banco sem tranca	Dados do leitor sempre recortados pelo usuário autenticado	apps/news/views.py:400, 407, 412 · apps/accounts/views.py:87, 109, 147 (request.user)
1. Banco sem tranca	Newsletter do leitor por e-mail + site da própria conta	apps/accounts/views.py:146-170
1. Banco sem tranca	Visão geral do painel só agrega o que a tela de destino liberaria	apps/common/newsroom/dashboard.py:74, 247, 322, 520-525 (is_item_visible)
2. Permissão no navegador	Portões do painel espelham exatamente o framework; o campo "painel" do login é conselho, não autorização	apps/accounts/panels.py:63-70, 202 · apps/accounts/panel_forms.py
2. Permissão no navegador	Regra editorial por notícia aplicada no servidor (GET e POST da edição e da restauração)	apps/news/wagtail_hooks.py:218-236 · apps/news/permissions.py:47-66
2. Permissão no navegador	Campos de publicação removidos no servidor para quem não publica	apps/news/wagtail_hooks.py:144-145, 162-163 · apps/news/wagtail_article.py:46-57
2. Permissão no navegador	Cópia de notícia não grava sobre a original (POST na cópia → 405)	apps/news/wagtail_article.py:216-227
2. Permissão no navegador	Ações em massa exigem permissão; exportação de e-mails só para superusuário e com neutralização de fórmulas	apps/news/admin.py:11-17, 74-81 · apps/news/wagtail_moderation.py:159-164
2. Permissão no navegador	SuperuserOnlyAdminMixin nos recursos guardados (códigos, identidades Google, vagas, equipe, curtidas)	apps/common/admin_mixins.py:119-135 · apps/accounts/admin.py:169, 208
2. Permissão no navegador	Login com Google não concede privilégio: conta nova nasce READER, identidade por sub, respeita bloqueio do axes	apps/accounts/oauth_views.py:104, 137-138, 240
2. Permissão no navegador	Restrições por papel no admin da escola aplicadas no queryset e nos campos	apps/school/admin.py:80-110, 298-311
3. IDOR	Excluir comentário exige posse	apps/news/views.py:515 (get_object_or_404(Comment, id=..., user=request.user))
3. IDOR	Comentar só em notícia publicada	apps/news/views.py:490
3. IDOR	Recuperação de senha amarrada à sessão, com estágio e prazo; reenvio usa o e-mail da sessão, nunca do POST	apps/accounts/code_views.py:127, 137-155, 265-286
3. IDOR	Descadastro da newsletter por token assinado (pk + e-mail)	apps/news/newsletter.py:43-61
3. IDOR	Currículos com nome UUID e locations internal no nginx	apps/hiring/models.py:12-18 · docker/nginx/nginx.conf:207-214
3. IDOR	Change view de Páginas só alcança objetos do queryset permitido ao papel	apps/school/admin.py:80-84
4. Chaves expostas	SECRET_KEY e credenciais do banco sem default: a aplicação não sobe sem elas	config/settings/base.py:12, 126-128

Categoria	Proteção verificada	Evidência (arquivo:linha)
4. Chaves expostas	.env, chaves e dumps fora do git e da imagem Docker	.gitignore:21-26, 51-62 · .dockerignore:1-4, 26-42
4. Chaves expostas	Histórico (182 commits) e bundle sem segredo real; site key do Turnstile injetada em runtime	git log -p --all · static/ · templates/components/turnstile_script.html:1
4. Chaves expostas	Tokens de redes sociais nunca reexibidos no admin	apps/social/admin.py:13-43, 100-115
4. Chaves expostas	Logs sem segredo (e-mail mascarado) e Sentry sem PII	apps/accounts/mailer.py:7-17, 46-61 · config/settings/production.py:55
4. Chaves expostas	Códigos de verificação guardados só como HMAC e ocultos no admin	apps/accounts/verification.py:100-119 · apps/accounts/admin.py:165
4. Chaves expostas	Turnstile falha fechado sem segredo em produção	apps/common/turnstile.py:26-32, 45-46
4. Chaves expostas	OAuth: redirect_uri fixo por env; state, nonce e PKCE na sessão do servidor	config/settings/base.py:412-415 · apps/accounts/oauth_views.py:177-183
5. XSS	Sanitização com bleach (allowlist de tags/atributos, protocolos http/https/mailto, iframe só YouTube, CSS filtrado) no save e no render	apps/common/sanitization.py:14-92 · apps/news/models.py:237 · apps/school/models.py:34 · apps/common/templatetags/sanitize.py
5. XSS	Autoescape em todos os templates; strings em JS via escapejs (JSON-LD, onclick, Alpine x-text)	templates/news/article_detail.html:43-54, 251, 299 · templates/school/*.html
5. XSS	JS próprio sem sinks de DOM (textContent/setAttribute; nenhum innerHTML/eval fora de vendor/)	static/newsroom/js/newsroom.js:323, 630
5. XSS	Uploads de imagem restritos e recodificados (Wagtail: jpg/png/webp, 5 MB, 25 MP; avatar vira JPEG)	config/settings/base.py:213-215 · apps/common/validators.py:34-63 · apps/accounts/models.py:63-70
5. XSS	Embeds e blocos reconstruídos a partir de dados estruturados (IDs por regex, tag de título não interpolada, URLBlock)	apps/common/embeds.py:32, 77 · templates/news/blocks/heading.html · templates/news/blocks/source.html
5. XSS	E-mails com autoescape e proteção contra header injection no assunto	apps/accounts/mailer.py:140 · templates/news/email/newsletter_article.html
5. XSS	Headers: HSTS, nosniff, frame-ancestors, X-Frame-Options DENY, cookies Secure/HttpOnly	config/settings/production.py:16-32 · docker/nginx/nginx.conf:150-167

2.2 Pontos fracos — riscos centrais

ALTA

A fronteira de superusuário existe só na interface: o cargo Administrador Geral a atravessa no servidor (PRIV-01) e em rota auxiliar (PRIV-02).

ALTA

Rotas por ID fora do padrão do projeto (on_site + publicada) vazam conteúdo editorial não publicado (IDOR-01, IDOR-02, ISO-01).

ALTA

Arquivos enviados são servidos como conteúdo ativo na mesma origem dos painéis, e a CSP não mitiga (XSS-01, XSS-02).

MÉDIA

Segredos dependem da disciplina do operador: nenhum check de startup barra os placeholders públicos (SEC-01).

2.3 Cobertura da auditoria — inventário de handlers

Todas as rotas de config/urls.py e dos apps, mais o admin do Django, o Wagtail e os blocos do nginx. "OK" significa verificado sem falha nas cinco categorias.

Área	Rotas / handlers	Resultado
Portal de notícias	GET /news/, /news/search/, /news/<slug>/, /category/, /tag/, /author/, /archive/, /htm/articles/	OK
Portal de notícias	GET /news/feed/, /news/category/<slug>/feed/, /sitemap.xml, /sitemap-<s>.xml	OK
Portal de notícias	POST /news/newsletter/subscribe/ · GET /news/newsletter/unsubscribe/<token>/	OK
Portal de notícias	GET /news/account/	IDOR-01, ISO-01
Portal de notícias	POST /news/toggle-bookmark/<id>/, /news/toggle-like/<id>/	IDOR-01, ISO-01
Portal de notícias	POST /news/comment/<id>/	ISO-01
Portal de notícias	POST /news/delete-comment/<id>/	OK
Portal de notícias	GET /news/newsletter/preview/<id>/	IDOR-02
Contas	/accounts/login/, logout/, register/, google/, google/callback/	OK
Contas	/accounts/confirmar-email/ (+ reenviar), profile/, delete-account/, toggle-newsletter/	OK
Contas	/accounts/password_reset/, senha/codigo/, senha/reenviar/, senha/nova/, reset/<uidb64>/<token>/	OK
Painel	/entrar/, /sair/, /sem-acesso/, /painel/, /painel/espaco/, /admin/guias/*	OK
Painel	/admin/accounts/customuser/* (usuários)	PRIV-01
Painel	/admin/media_library/mediafile/add/ · /cms/documents/add/	XSS-01
Painel	Demais ModelAdmins (contact, school, social, news, hiring, accounts auditoria)	OK
Painel	/cms/ (snippets de notícia, moderação, relatório "em revisão", configurações)	OK
Painel	/hiring/application/<id>/resume/	PRIV-02
Escola e utilidades	/, /sobre/, /privacidade/, /cursos/<slug>/, /<slug>/, /team/, /contact/	OK
Escola e utilidades	/healthz/, /robots.txt, /i18n/, /documents/<id>/<arquivo>	OK
Infra (nginx)	location /media/	XSS-01
Infra (nginx)	location /protected/ e /media/hiring/resumes/ (internal)	OK

3. Achados detalhados por categoria

Cada categoria traz o mecanismo equivalente na stack, a tabela de achados e, em seguida, o detalhe de cada um (trecho real do código, exploração, condição e forma de verificação).

3.1 Banco sem tranca — isolamento de inquilino/dono

Mecanismo na stack: Django + PostgreSQL, sem RLS. O isolamento é feito na aplicação: Sites framework (FK site + CurrentSiteManager "on_site", SITE_ID=1 fixo), recorte por dono (request.user) e permissões de modelo por cargo.

Severidade	Arquivo:linha	Descrição
BAIXA	apps/news/views.py:398-409 apps/news/views.py:438, 464, 490 apps/news/views.py:533	ISO-01 — Rotas do leitor carregam notícias com Article.objects, fora do isolamento por Site (on_site) O mecanismo de isolamento do projeto é o Sites framework (FK site + CurrentSiteManager on_site; README: "Views públicas usam Model.on_site, nunca Model.objects"). Estas rotas buscam ou listam notícias sem o Site atual. Condição: Só explorável se um segundo Site for ativado.

ISO-01	BAIXA	Rotas do leitor carregam notícias com Article.objects, fora do isolamento por Site (on_site)
--------	-------	---

Onde: apps/news/views.py:398-409 — user_dashboard (favoritos e curtidas); apps/news/views.py:438, 464, 490 — toggle_bookmark, toggle_like, add_comment; apps/news/views.py:533 — newsletter_preview

Por que é explorável: Hoje não há segundo Site ativo (SITE_ID=1 fixo), então não há vazamento entre portais. Com multi-site real, leitores de um portal passam a interagir com — e ler — notícias do outro.

Condição de explorabilidade: Só explorável se um segundo Site for ativado.

Como foi verificado: Estática (código + settings + README).

```
# apps/news/views.py:490 (a mais segura das três: tem status, falta o site)
article = get_object_or_404(Article, id=article_id, status=Article.Status.PUBLISHED)
```

3.2 Permissão definida no navegador

Mecanismo na stack: Não há SPA: o "frontend" são templates Django, Unfold e Wagtail. Gates de interface: navegação do painel (navigation.py), fieldsets do admin, rótulos "só superusuário". No servidor: ModelAdmin.has_*_permission, permission policies do Wagtail e decorators.

Severidade	Arquivo:linha	Descrição
ALTA	apps/accounts/admin.py:113-114 apps/accounts/admin_roles.py:56-66, 126-138, 195-196 apps/accounts/admin.py:116, 132 apps/common/newsroom/navigation.py:62-65 apps/common/admin_mixins.py:119-135	PRIV-01 — "Administrador Geral" se promove a superusuário (e troca a senha de superusuários) pelo admin O projeto trata superusuário como fronteira própria (telas guardadas, códigos de verificação, identidades Google, candidaturas, exportação de e-mails), mas o formulário de usuário aceita is_superuser, groups e user_permissions de qualquer conta com accounts.change_customuser — que o cargo Administrador Geral recebe. Condição: Conta com cargo Administrador Geral (ou qualquer conta com accounts.change_customuser), interna ou comprometida.
MÉDIA	apps/hiring/views.py:12-14, 20 apps/hiring/admin.py:117-118 apps/common/newsroom/navigation.py:238-239 apps/accounts/admin_roles.py:56-66	PRIV-02 — Download de currículos ignora a restrição "só superusuário" das Candidaturas Candidaturas são "recurso guardado", visível só a superusuário no admin e no menu, mas a rota de download confere apenas a permissão de modelo, que o grupo Administrador Geral recebe. Condição: Conta staff com hiring.view_application (Administrador Geral). Afeta os currículos já recebidos.

PRIV-01

ALTA

"Administrador Geral" se promove a superusuário (e troca a senha de superusuários) pelo admin

Onde: apps/accounts/admin.py:113-114 — fieldset "Cargo e permissões" expõe is_superuser, groups e user_permissions; a classe (:66-160) não tem get_readonly_fields/get_fieldsets por papel; apps/accounts/admin_roles.py:56-66, 126-138, 195-196 — grupo "Administrador Geral" recebe todas as permissões de accounts e de auth.group; apps/accounts/admin.py:116, 132 · apps/common/newsroom/navigation.py:62-65 · apps/common/admin_mixins.py:119-135 — a fronteira "superusuário é escolha separada" existe só como texto e navegação

Por que é explorável: POST em /admin/accounts/customuser/<próprio id>/change/ com is_superuser=on promove a própria conta. O mesmo papel troca a senha de um superusuário em /admin/accounts/customuser/<id>/password/ (UserAdmin.user_change_password só exige permissão de change) e edita grupos (auth.change_group).

Condição de explorabilidade: Conta com cargo Administrador Geral (ou qualquer conta com accounts.change_customuser), interna ou comprometida.

Como foi verificado: Dinâmica: sonda com usuário Administrador Geral → POST 302 e is_superuser=True no banco.

```
# apps/accounts/admin.py:113-114
('Cargo e permissões', {
    'fields': ('role', 'is_active', 'is_staff', 'is_superuser', 'groups', 'user_permissions'),

# apps/accounts/admin_roles.py:126-138 (GENERAL_ADMIN_APP_LABELS, :56-66, inclui 'accounts')
def _general_admin_permissions():
    permissions = list(
        Permission.objects.filter(
            content_type__app_label__in=GENERAL_ADMIN_APP_LABELS,
        ).select_related('content_type')
    )
    permissions.extend(
        Permission.objects.filter(
            content_type__app_label='auth',
            content_type__model='group',
        ).select_related('content_type')
    )
    return permissions
```

PRIV-02

MÉDIA

Download de currículos ignora a restrição "só superusuário" das Candidaturas

Onde: apps/hiring/views.py:12-14, 20 — @staff_member_required + permission_required("hiring.view_application"); busca a candidatura só pelo ID; apps/hiring/admin.py:117-118 · apps/common/newsroom/navigation.py:238-239 — o admin (SuperuserOnlyAdminMixin) e a navegação restringem Candidaturas a superusuário; apps/accounts/admin_roles.py:56-66 — 'hiring' está em GENERAL_ADMIN_APP_LABELS

Por que é explorável: Um Administrador Geral (não superusuário) enumera /hiring/application/1..N/resume/ (ID sequencial) e baixa currículos — dados pessoais —, enquanto /admin/hiring/application/ responde 403.

Condição de explorabilidade: Conta staff com hiring.view_application (Administrador Geral). Afeta os currículos já recebidos.

Como foi verificado: Dinâmica: changelist 403; download 200 com X-Accel-Redirect para /protected/hiring/resumes/.

```
# apps/hiring/views.py:12-20
@staff_member_required
@permission_required('hiring.view_application', raise_exception=True)
def download_resume(request, application_id):
    ...
    application = get_object_or_404(Application, pk=application_id)
```

3.3 IDOR — acesso a objeto por ID sem checar posse

Mecanismo na stack: Rotas com <int:...>, <slug:...> e <path:token> em apps/*/urls.py, change views do Django admin e viewsets do Wagtail.

Severidade	Arquivo:linha	Descrição
ALTA	apps/news/views.py:436-438 apps/news/views.py:462-464 apps/news/views.py:398-409 templates/news/account/dashboard.html:105-106	IDOR-01 — Favoritar/curtir por ID aceita notícia não publicada e o painel do leitor exibe o conteúdo Os toggles buscam a notícia só pelo ID, sem status=PUBLISHED nem on_site. O painel lista os favoritos sem filtro de status e, sem resumo, renderiza o conteúdo inteiro (o line-clamp é só CSS; o texto está no HTML). Condição: Conta de leitor (auto-cadastro) com e-mail confirmado; IDs sequenciais.
MÉDIA	apps/news/views.py:524-536	IDOR-02 — Prévia de newsletter renderiza qualquer notícia (inclusive rascunho) para qualquer staff A rota só exige is_staff: não confere news.view_article, status nem site. O Administrador Komuniki (staff, sem permissão de notícias) lê qualquer notícia por ID; já Editor e Repórter (não staff) não alcançam a rota. Condição: Qualquer conta is_staff. A rota não é linkada na interface (só por URL).

IDOR-01 **ALTA** Favoritar/curtir por ID aceita notícia não publicada e o painel do leitor exibe o conteúdo

Onde: apps/news/views.py:436-438 — toggle_bookmark: get_object_or_404(Article, id=article_id); apps/news/views.py:462-464 — toggle_like: mesma busca, sem status nem site; apps/news/views.py:398-409 — user_dashboard lista Article.objects sem filtro de status; templates/news/account/dashboard.html:105-106 — renderiza título e excerpt|default:article.content

Por que é explorável: Um leitor com cadastro público e e-mail confirmado faz POST em /news/toggle-bookmark/<n>/ para n=1..N e lê rascunhos, agendadas e arquivadas em /news/account/ — conteúdo editorial sob embargo.

Condição de explorabilidade: Conta de leitor (auto-cadastro) com e-mail confirmado; IDs sequenciais.

Como foi verificado: Dinâmica: rascunho dá 404 na URL pública, mas título e conteúdo aparecem no painel após o POST.

```
# apps/news/views.py:436-438
def toggle_bookmark(request, article_id):
    article = get_object_or_404(Article, id=article_id)

# apps/news/views.py:398-409
saved_articles = (
    Article.objects
    .filter(bookmarks__user=user)
```

IDOR-02 **MÉDIA** Prévia de newsletter renderiza qualquer notícia (inclusive rascunho) para qualquer staff

Onde: apps/news/views.py:524-536 — @staff_member_required; get_object_or_404(Article, id=article_id) em :533

Por que é explorável: GET /news/newsletter/preview/<n>/ percorrendo IDs sequenciais.

Condição de explorabilidade: Qualquer conta is_staff. A rota não é linkada na interface (só por URL).

Como foi verificado: Dinâmica: Administrador Komuniki sem news.view_article recebeu 200 com o título do rascunho.

```
# apps/news/views.py:524-533
@staff_member_required
def newsletter_preview(request, article_id):
    ...
    article = get_object_or_404(Article, id=article_id)
```

3.4 Chaves expostas (hardcode)

Mecanismo na stack: Settings via django-environ (.env/.env.prod), Docker Compose, GitHub Actions, scripts de deploy e docs. O "bundle" do frontend é static/ servido por WhiteNoise/nginx (sem build JS).

Severidade	Arquivo:linha	Descrição
MÉDIA	<code>.env.prod.example:11</code> <code>.env.prod.example:25, 28, 39</code> <code>.env.example:2, 10</code> <code>scripts/deploy/kellysys-deploy:137</code> <code>apps/accounts/checks.py</code>	SEC-01 — Placeholders públicos de segredos são aceitos em produção (sem validação de startup) config/settings/base.py:12 exige SECRET_KEY (bom), mas nada rejeita os valores de exemplo publicados no repositório: o Django só emite o aviso security.W009 e o deploy usa --fail-level ERROR. Condição: Operador não sobrescrever o placeholder.
BAIXA	<code>docker/docker-compose.yml:39, 41</code> <code>docker/docker-compose.yml:52-53</code>	SEC-02 — Compose de desenvolvimento publica Postgres (senha fixa) e Mailpit em todas as interfaces Publicar em 0.0.0.0 expõe o banco de desenvolvimento (com a senha do repositório) e a caixa de e-mails capturados — onde chegam códigos de verificação e de recuperação de senha — a quem estiver na mesma rede. Condição: Máquina de desenvolvimento acessível na rede (Wi-Fi compartilhado, VM na nuvem). Produção não publica o banco.
INFORMATIVA	<code>.env.prod.example:14</code> <code>docs/technical/cloudflare-bots.md:56, 86, 103</code>	SEC-03 — IP de origem da VPS publicado no repositório Não é segredo criptográfico, mas revela a origem atrás do Cloudflare. O próprio documento (:86) avisa que sem o firewall um atacante ignora o Bot Fight Mode batendo direto no IP. Condição: Relevante se scripts/deploy/cloudflare-firewall.sh não estiver aplicado e o repositório for público.
INFORMATIVA	<code>commit a3b8544 → 5033972</code> <code>config/settings/design_preview.py:5, 38-39</code> <code>.github/workflows/django.yml:19, 24</code> <code>deploy-production.yml:27, 32</code>	SEC-04 — Histórico e configs de preview/CI com chaves fixas não-produtivas A varredura de 182 commits não achou segredo real (tokens de API, chaves privadas, DSN, senhas de produção). Os valores listados são de dev/teste; o default inseguro da SECRET_KEY existiu por 2 dias e foi removido. Condição: Ambiente iniciado entre 23 e 25/02/2026 sem SECRET_KEY → rotacionar.

SEC-01

MÉDIA

Placeholders públicos de segredos são aceitos em produção (sem validação de startup)

Onde: `.env.prod.example:11` — SECRET_KEY=__TROQUE_POR_UMA_CHAVE_FORTE__; `.env.prod.example:25, 28, 39` — POSTGRES_PASSWORD / DB_PASSWORD / EMAIL_HOST_PASSWORD de exemplo; `.env.example:2, 10` — SECRET_KEY=your-secret-key-here, DB_PASSWORD=news_portal_pass; `scripts/deploy/kellysys-deploy:137` — check --deploy --fail-level ERROR (security.W009 é só aviso); `apps/accounts/checks.py` — o único system check próprio valida e-mail, não segredos

Por que é explorável: Se o `.env.prod` for criado a partir do exemplo sem trocar a chave, a SECRET_KEY é pública: dá para forjar valores assinados — tokens de descadastro (`apps/news/newsletter.py:43-61`) e cookies de mensagens — e o pepper HMAC dos códigos de verificação (`apps/accounts/verification.py:100-119`) deixa de ser segredo. Sessões em banco e o reset atrelado ao hash da senha limitam o impacto.

Condição de explorabilidade: Operador não sobrescrever o placeholder.

Como foi verificado: Dinâmica: `manage.py check --deploy --fail-level ERROR` com o placeholder terminou com exit 0.

```
# .env.prod.example:11
SECRET_KEY=__TROQUE_POR_UMA_CHAVE_FORTE__

# scripts/deploy/kellysys-deploy:137
compose run --rm web python manage.py check --deploy --fail-level ERROR
```

SEC-02

BAIXA

Compose de desenvolvimento publica Postgres (senha fixa) e Mailpit em todas as interfaces

Onde: docker/docker-compose.yml:39, 41 — POSTGRES_PASSWORD: news_portal_pass; ports "5432:5432"; docker/docker-compose.yml:52-53 — Mailpit "8025:8025" e "1025:1025"

Por que é explorável: Qualquer host na rede conecta em <ip-do-dev>:5432 com news_portal_user/news_portal_pass ou abre :8025.

Condição de explorabilidade: Máquina de desenvolvimento acessível na rede (Wi-Fi compartilhado, VM na nuvem). Produção não publica o banco.

Como foi verificado: Estática.

```
# docker/docker-compose.yml:39-41
  POSTGRES_PASSWORD: news_portal_pass
  ports:
    - "5432:5432"
```

SEC-03

INFORMATIVA

IP de origem da VPS publicado no repositório

Onde: .env.prod.example:14 — ALLOWED_HOSTS=...,2.25.178.16,...; docs/technical/cloudflare-bots.md:56, 86, 103 — IP citado nos passos de DNS e de teste

Por que é explorável: Ataque direto à origem, contornando WAF/rate limit do Cloudflare.

Condição de explorabilidade: Relevante se scripts/deploy/cloudflare-firewall.sh não estiver aplicado e o repositório for público.

Como foi verificado: Estática.

```
# .env.prod.example:14
ALLOWED_HOSTS=komuniki.com.br,www.komuniki.com.br,kellyfarias.com.br,www.kellyfarias.com.br,2.25.178.16,...
```

SEC-04

INFORMATIVA

Histórico e configs de preview/CI com chaves fixas não-produtivas

Onde: commit a3b8544 → 5033972 — config/settings/base.py:12 tinha default='django-insecure-change-me-in-production' entre 23 e 25/02/2026; config/settings/design_preview.py:5, 38-39 — SECRET_KEY de preview (loopback) e chaves de teste públicas da Cloudflare; .github/workflows/django.yml:19, 24 · deploy-production.yml:27, 32 — credenciais do Postgres efêmero do CI

Por que é explorável: Só se algum ambiente tiver subido naquele intervalo sem SECRET_KEY definida.

Condição de explorabilidade: Ambiente iniciado entre 23 e 25/02/2026 sem SECRET_KEY → rotacionar.

Como foi verificado: git log -p --all em 182 commits (clone não-raso).

```
# git show a3b8544:config/settings/base.py (linha 12)
SECRET_KEY = env('SECRET_KEY', default='django-insecure-change-me-in-production')
```

3.5 Inputs sem tratamento (XSS)

Mecanismo na stack: Templates Django com autoescape + HTMX + Alpine.js; HTML de usuário via bleach (sanitize_html) e StreamField do Wagtail; uploads servidos pelo nginx em /media/; e-mails via render_to_string.

Severidade	Arquivo:linha	Descrição
ALTA	apps/media_library/admin.py:27, 97-103 apps/media_library/models.py:37 config/settings/base.py:200-201 docker/nginx/nginx.conf:216-219, 167	XSS-01 — Upload de HTML/SVG (mídia e documentos) servido inline na mesma origem dos painéis — XSS armazenado A Biblioteca de mídia (Django admin) e os Documentos do Wagtail aceitam .html e .svg. O nginx entrega /media/... com o Content-Type do mime.types (text/html, image/svg+xml), sem Content-Disposition: attachment, e o bloco /media/ (só "expires") herda do server a CSP que permite script inline. A view /documents/ do Wagtail aplica sandbox, mas o arquivo bruto em /media/documents/ não passa por ela. Condição: Conta com permissão de upload + vítima privilegiada abrir o link.
BAIXA	config/settings/base.py:454-459 docker/nginx/nginx.conf:167	XSS-02 — CSP com 'unsafe-inline' e 'unsafe-eval' em script-src anula a defesa em profundidade Exigida hoje pelo build padrão do Alpine.js (usa new Function) e por scripts inline. Qualquer injeção que escape do autoescape vira execução — é o que torna XSS-01 executável. Condição: Depende de outra falha de injeção.

Severidade	Arquivo:linha	Descrição
INFORMATIVA	templates/accounts/password_reset/new_password.html:44	XSS-03 — safe residual no template de senha nova O help_text vem dos validadores de senha do Django (texto do servidor, já marcado como seguro), não de usuário: não é explorável hoje. Contraria a regra do projeto ("nunca safe") e vira XSS se um help_text passar a incluir dado dinâmico. Condição: Help text com dado de usuário no futuro.

XSS-01 **ALTA** Upload de HTML/SVG (mídia e documentos) servido inline na mesma origem dos painéis — XSS armazenado

Onde: apps/media_library/admin.py:27, 97-103 — .svg tratado como imagem; clean_file aceita qualquer extensão; apps/media_library/models.py:37 — FileField sem validador de extensão; config/settings/base.py:200-201 — Documentos do Wagtail sem WAGTAILDOCS_EXTENSIONS (aceita qualquer extensão); docker/nginx/nginx.conf:216-219, 167 — /media/ servida direto; herda a CSP com script-src 'unsafe-inline'

Por que é explorável: Administrador Komuniki (mídia) ou Editor de Notícias (documentos) envia evil.html; um superusuário que abrir o link em komuniki.com.br/media/... executa JS na mesma origem do /admin/ e do /cms/, lê o csrftoken e cria ou promove contas — escalada até superusuário.

Condição de explorabilidade: Conta com permissão de upload + vítima privilegiada abrir o link.

Como foi verificado: Dinâmica: .html e .svg aceitos pela Biblioteca de mídia (POST no admin → 302) e .html aceito pela validação do modelo de Documento (full_clean); estática para o nginx (regra de herança do add_header).

```
# apps/media_library/admin.py:97-103
def clean_file(self):
    uploaded = self.cleaned_data.get('file')
    # ... a biblioteca aceita qualquer arquivo.
    if uploaded and Path(uploaded.name).suffix.lower() in ALLOWED_IMAGE_EXTENSIONS:
        validate_uploaded_image(uploaded)
    return uploaded

# docker/nginx/nginx.conf:216-219
location /media/ {
    alias /app/media/;
    expires 7d;
}
```

XSS-02 **BAIXA** CSP com 'unsafe-inline' e 'unsafe-eval' em script-src anula a defesa em profundidade

Onde: config/settings/base.py:454-459 — script-src: SELF, UNSAFE_INLINE, UNSAFE_EVAL, challenges.cloudflare.com; docker/nginx/nginx.conf:167 — mesma política no header do nginx

Por que é explorável: Não é explorável sozinha; amplifica qualquer outra injeção.

Condição de explorabilidade: Depende de outra falha de injeção.

Como foi verificado: Estática.

```
# config/settings/base.py:454-459
'script-src': [
    SELF,
    UNSAFE_INLINE,
    UNSAFE_EVAL,
    'https://challenges.cloudflare.com',
],
```

XSS-03 **INFORMATIVA** |safe residual no template de senha nova

Onde: templates/accounts/password_reset/new_password.html:44 — {{ field.help_text|safe }}

Por que é explorável: Não explorável no estado atual.

Condição de explorabilidade: Help text com dado de usuário no futuro.

Como foi verificado: Estática.

```
{# templates/accounts/password_reset/new_password.html:44 #}
<div class="mt-1 text-xs text-slate-500 font-ui">{{ field.help_text|safe }}</div>
```

4. Recomendações priorizadas

Ordem sugerida de execução. P1 fecha os três caminhos de alta severidade; cada item vira uma issue na seção 5.

Prioridade	Ações	Achados
P1	Imediato (esta semana) • Tornar <code>is_superuser</code>, <code>groups</code> e <code>user_permissions</code> somente leitura para não-superusuários; negar a não-superusuários editar, excluir ou trocar a senha de superusuários; tirar <code>add/change/delete</code> de <code>auth.group</code> do Administrador Geral. • Allowlist de extensões na Biblioteca de mídia e <code>WAGTAILDOCS_EXTENSIONS</code>; no <code>nginx</code>, bloquear <code>HTML/SVG/XML/JS</code> em <code>/media/</code>, tornar <code>/media/documents/</code> <code>internal</code> e forçar <code>attachment + CSP sandbox</code>; varrer os arquivos já enviados. • Toggles de favorito/curtida e painel do leitor com <code>Article.on_site + status=PUBLISHED</code>.	PRIV-01, XSS-01, IDOR-01
P2	Curto prazo (até 2 semanas) • Download de currículo com a mesma regra do <code>ApplicationAdmin</code> (superusuário) — ou retirar <code>hire</code> do Administrador Geral. • Prévia de newsletter com <code>news.view_article + Article.on_site</code>, ou remover a rota (não é linkada). • System check de deploy que rejeita placeholders e <code>SECRET_KEY</code> fraca como Error.	PRIV-02, IDOR-02, SEC-01
P3	Médio prazo (próximo ciclo) • Padronizar <code>Article.on_site</code> em todas as rotas do leitor e adicionar teste que falhe se views públicas usarem <code>Article.objects</code>. • CSP sem <code>'unsafe-eval'</code> (build CSP do Alpine) e sem <code>'unsafe-inline'</code> em <code>script-src</code> (nonce ou arquivos estáticos); remover o <code> safe</code> residual.	ISO-01, XSS-02, XSS-03
P4	Higiene contínua • Portas do compose de dev só em <code>127.0.0.1</code>; retirar o IP de origem do repositório e confirmar o firewall do Cloudflare. • Secret scanning (ex.: gitleaks) no CI; rotacionar <code>SECRET_KEY</code> se algum ambiente subiu entre 23 e 25/02/2026. • Transformar as sondas desta auditoria em testes de regressão (com as asserções invertidas).	SEC-02, SEC-03, SEC-04

5. ISSUES PARA O GITHUB

Texto completo de cada issue, em Markdown, pronto para copiar e colar. Copie tudo o que está entre as linhas `--- ISSUE n ---` e `--- FIM ISSUE n ---`: a primeira linha é o título e a segunda, as labels sugeridas. Achados triviais do mesmo tema foram agrupados para não gerar issues demais. Alguns leitores de PDF descartam linhas em branco ao copiar; o Markdown abaixo continua válido sem elas, e o mesmo texto está em docs/security-audit/issues/issue-NN.md.

#	Título	Achados
1	[Segurança] "Administrador Geral" consegue se promover a superusuário pelo admin	PRIV-01
2	[Segurança] Favoritar/curtir por ID expõe notícias não publicadas no painel do leitor	IDOR-01 + ISO-01
3	[Segurança] Upload de HTML/SVG em mídia e documentos permite XSS armazenado	XSS-01
4	[Segurança] Download de currículos ignora a restrição a superusuário das Candidaturas	PRIV-02
5	[Segurança] Prévia de newsletter mostra qualquer notícia (inclusive rascunho) a qualquer staff	IDOR-02
6	[Segurança] Placeholders de segredos passam no deploy (+ defaults de dev e IP de origem)	SEC-01 + SEC-02 + SEC-03 + SEC-04
7	[Segurança] Endurecer a CSP de script-src e remover o safe residual	XSS-02 + XSS-03

Issue 1 — PRIV-01

--- ISSUE 1 ---

Título: [Segurança] "Administrador Geral" consegue se promover a superusuário pelo admin
Labels sugeridas: security, severidade: alta

Descrição

O projeto trata superusuário como uma fronteira separada: recursos guardados (códigos de verificação, identidades Google, candidaturas, exportação de e-mails) são liberados só para ``is_superuser``, e o próprio formulário diz "Superusuário continua sendo uma escolha separada". No servidor, porém, o ``CustomUserAdmin`` aceita ``is_superuser``, ``groups`` e ``user_permissions`` de qualquer conta com ``accounts.change_customuser`` — permissão que o grupo `**Administrador Geral**` recebe junto com todas as de ``auth.group``.

Por que é explorável

Um Administrador Geral (não superusuário) envia ``POST /admin/accounts/customuser/<próprio id>/change/`` com ``is_superuser=on`` e vira superusuário. Com a mesma permissão ele troca a senha de um superusuário existente em ``/admin/accounts/customuser/<id>/password/`` (o ``UserAdmin`` só exige permissão de change) e edita grupos no ``/admin/auth/group/`` e no ``/cms/groups/``.

Confirmado com teste dinâmico (Django test client): o POST retornou 302 e ``is_superuser`` ficou ``True`` no banco.

Evidência

```
- `apps/accounts/admin.py:113-114`
```python
('Cargo e permissões', {
 'fields': ('role', 'is_active', 'is_staff', 'is_superuser', 'groups', 'user_permissions'),
 ...
- `apps/accounts/admin_roles.py:56-66` (`GENERAL_ADMIN_APP_LABELS` inclui `accounts`) e `:126-138`
```python
def _general_admin_permissions():
    permissions = list(
        Permission.objects.filter(
            content_type__app_label__in=GENERAL_ADMIN_APP_LABELS,
        ).select_related('content_type')
    )
    permissions.extend(
        Permission.objects.filter(
            content_type__app_label='auth',
            content_type__model='group',
        ).select_related('content_type')
    )
    return permissions
...
- Fronteira só na interface: `apps/accounts/admin.py:116, 132`,
```

```
`apps/common/newsroom/navigation.py:62-65`, `apps/common/admin_mixins.py:119-135`.
```

Impacto
Escalada de privilégio de uma conta administrativa intermediária (ou de uma conta dessas comprometida) para controle total: superusuário, tomada de contas de outros superusuários e acesso a tudo o que hoje é "só superusuário".

Sugestão de correção
```python  
# apps/accounts/admin.py – CustomUserAdmin  
SUPERUSER\_ONLY\_FIELDS = ('is\_superuser', 'groups', 'user\_permissions')  
  
def get\_readonly\_fields(self, request, obj=None):  
 fields = list(super().get\_readonly\_fields(request, obj))  
 if not request.user.is\_superuser:  
 fields += [f for f in self.SUPERUSER\_ONLY\_FIELDS if f not in fields]  
 return fields  
  
def has\_change\_permission(self, request, obj=None):  
 if obj is not None and obj.is\_superuser and not request.user.is\_superuser:  
 return False # cobre também a troca de senha (user\_change\_password)  
 return super().has\_change\_permission(request, obj)  
  
def has\_delete\_permission(self, request, obj=None):  
 if obj is not None and obj.is\_superuser and not request.user.is\_superuser:  
 return False  
 return super().has\_delete\_permission(request, obj)  
...`

Em `apps/accounts/admin\_rolas.py`, dar ao Administrador Geral só `auth.view\_group` (sem add/change/delete), para que ele não consiga conceder permissões a si mesmo pelos grupos. A sincronização cargo → grupo (`save\_related`) continua funcionando.

**## Critérios de aceite**

- [ ] Administrador Geral não superusuário vê `is\_superuser`, `groups` e `user\_permissions` como somente leitura, e um POST forjado com `is\_superuser=on` não altera o valor (teste automatizado).
- [ ] Não superusuário recebe 403 ao abrir, editar, trocar a senha ou excluir um superusuário.
- [ ] Administrador Geral não consegue criar/editar grupos em `/admin/auth/group/` nem em `/cms/groups/`.
- [ ] Testes de regressão em `apps/accounts/` cobrindo os três vetores; suíte completa verde.

--- FIM ISSUE 1 ---

## Issue 2 — IDOR-01 + ISO-01

--- ISSUE 2 ---

Título: [Segurança] Favoritar/curtir por ID expõe notícias não publicadas no painel do leitor  
Labels sugeridas: security, severidade: alta

**## Descrição**  
`toggle\_bookmark` e `toggle\_like` buscam a notícia só pelo ID (`get\_object\_or\_404(Article, id=article\_id)`), sem `status=PUBLISHED` nem `Article.on\_site`. O painel do leitor (`/news/account/`) lista os favoritos com `Article.objects`, sem filtro de status, e renderiza o título e – quando não há resumo – o conteúdo inteiro da notícia (o `line-clamp` é só CSS).

Relacionado (baixa): as mesmas rotas e `add\_comment` fogem do isolamento por Site que o projeto adota ("Views públicas usam `Model.on\_site`, nunca `Model.objects`").

**## Por que é explorável**  
Qualquer pessoa cria conta de leitor, confirma o e-mail e faz `POST /news/toggle-bookmark/<n>/` para n = 1..N (IDs sequenciais). Rascunhos, agendadas e arquivadas aparecem em `/news/account/` com título e texto.

Confirmado com teste dinâmico: o rascunho dá 404 na URL pública, mas título e conteúdo aparecem no painel após o POST.

**## Evidência**

```
- `apps/news/views.py:436-438` e `:462-464`
```python  
def toggle_bookmark(request, article_id):  
    article = get_object_or_404(Article, id=article_id)  
...  
- `apps/news/views.py:398-409` (`saved_articles = Article.objects.filter(bookmarks__user=user)`, sem status)  
- `templates/news/account/dashboard.html:105-106` (`{{ article.title }}`, `{{ article.excerpt|default:article.content|striptags }}`)
```

```
- Isolamento por Site: `apps/news/views.py:438, 464, 490, 398-409`

## Impacto
Vazamento de conteúdo editorial não publicado (matérias sob embargo, agendadas ou retiradas do ar)
para qualquer leitor. Com multi-site ativado, também mistura dados entre os portais.

## Sugestão de correção
```python
def _published_article_or_404(article_id):
 return get_object_or_404(Article.on_site, id=article_id, status=Article.Status.PUBLISHED)

toggle_bookmark / toggle_like / add_comment
article = _published_article_or_404(article_id)

user_dashboard
saved_articles = Article.on_site.filter(status=Article.Status.PUBLISHED, bookmarks__user=user)...
liked_articles = Article.on_site.filter(status=Article.Status.PUBLISHED, likes__user=user)...
```

## Critérios de aceite
- [ ] POST em `/news/toggle-bookmark/<id>/` e `/news/toggle-like/<id>/` para notícia em rascunho,
arquivada ou de outro Site retorna 404 e não cria registro.
- [ ] `/news/account/` lista só notícias publicadas do Site atual, mesmo que existam favoritos
antigos para outras.
- [ ] `add_comment` usa `Article.on_site`.
- [ ] Testes cobrindo rascunho e notícia de outro Site.

--- FIM ISSUE 2 ---
```

Issue 3 — XSS-01

--- ISSUE 3 ---

Título: [Segurança] Upload de HTML/SVG em mídia e documentos permite XSS armazenado
Labels sugeridas: security, severidade: alta

Descrição
A Biblioteca de mídia (Django admin) aceita qualquer extensão — inclusive `.html` e `.svg` — e os Documentos do Wagtail não têm `WAGTAILDOCS\_EXTENSIONS`. O nginx serve `/media/` direto, com o Content-Type do `mime.types` (`text/html`, `image/svg+xml`), sem `Content-Disposition: attachment`, e o bloco `/media/` (que só usa `expires`) herda do `server` a CSP com `script-src 'unsafe-inline'`. A view `/documents/` do Wagtail aplica sandbox, mas o arquivo bruto em `/media/documents/` não passa por ela.

Por que é explorável
Um Administrador Komuniki (mídia) ou Editor de Notícias (documentos) envia `evil.html`. Um superusuário que abrir o link em `https://komuniki.com.br/media/...` executa JavaScript na mesma origem do `/admin/` e do `/cms/`: o script lê o `csrftoken` de uma página do painel e cria ou promove contas.

Confirmado com teste dinâmico: `.html` e `.svg` foram aceitos na Biblioteca de mídia (POST no admin → 302) e `.html` passou na validação do modelo de Documento do Wagtail (`full\_clean`).

Evidência

```
- `apps/media_library/admin.py:97-103`
```python
def clean_file(self):
 uploaded = self.cleaned_data.get('file')
 # ... a biblioteca aceita qualquer arquivo.
 if uploaded and Path(uploaded.name).suffix.lower() in ALLOWED_IMAGE_EXTENSIONS:
 validate_uploaded_image(uploaded)
 return uploaded
```

- `apps/media_library/admin.py:27` (`.svg` tratado como imagem), `apps/media_library/models.py:37`
- `config/settings/base.py:200-201` (sem `WAGTAILDOCS_EXTENSIONS`)
- `docker/nginx/nginx.conf:216-219` (location /media/) e `:167` (CSP herdada)
```

Impacto
XSS armazenado na origem dos painéis → escalada de uma conta com upload até superusuário (ação em nome de quem abrir o link).

Sugestão de correção

- Allowlist na Biblioteca de mídia (validador no modelo e no form); sem `.svg`, `.html`, `.htm`, `.xhtml`, `.xml`, `.js`.
- `WAGTAILDOCS_EXTENSIONS = ['pdf', 'doc', 'docx', 'odt', 'rtf', 'txt', 'csv', 'xls', 'xlsx', 'ppt', 'pptx']`.
- nginx: `location /media/documents/ { internal; }` (documentos só pela view do Wagtail) e bloquear

```
tipos ativos com `location ~* ^/media/.+\. (html?|xhtml|svg|xml|js)$ { return 404; }`. Se o bloco
`/media/` ganhar `add_header` próprio (ex.: `Content-Disposition: attachment`, `CSP: sandbox`),
repetir nele HSTS/nosniff — `add_header` num location cancela a herança do `server`.
4. Varrer os volumes: `find /app/media -iregex '.*\.(html?|xhtml|svg|xml|js)\. '` e remover o que
houver.
5. Médio prazo: servir uploads de um domínio próprio, sem cookies dos painéis.

## Critérios de aceite
- [ ] Upload de `.html`, `.htm`, `.svg`, `.xml` e `.js` é recusado na Biblioteca de mídia e em
  Documentos (teste automatizado).
- [ ] `GET /media/documents/<arquivo>` não é servido diretamente; documentos saem só por
  `/documents/<id>/<nome>`.
- [ ] `curl -I https://<domínio>/media/<arquivo>.html` não devolve `text/html` executável (404 ou
  attachment + sandbox).
- [ ] Varredura dos volumes de produção sem arquivos ativos remanescentes.

--- FIM ISSUE 3 ---
```

Issue 4 — PRIV-02

--- ISSUE 4 ---

Título: [Segurança] Download de currículos ignora a restrição a superusuário das Candidaturas
Labels sugeridas: security, severidade: média

Descrição

Candidaturas são "recurso guardado": o `ApplicationAdmin` usa `SuperuserOnlyAdminMixin` e o menu só as mostra a superusuário. A rota de download, porém, confere só `hiring.view\_application`, permissão que o grupo Administrador Geral recebe (`hiring` está em `GENERAL\_ADMIN\_APP\_LABELS`).

Por que é explorável

Um Administrador Geral (não superusuário) percorre `/hiring/application/1..N/resume/` (ID sequencial) e baixa os currículos, enquanto `/admin/hiring/application/` responde 403.

Confirmado com teste dinâmico: changelist 403; download 200 com `X-Accel-Redirect: /protected/hiring/resumes/...`.

Evidência

```
- `apps/hiring/views.py:12-20`
``python
@staff_member_required
@permission_required('hiring.view_application', raise_exception=True)
def download_resume(request, application_id):
    ...
    application = get_object_or_404(Application, pk=application_id)
...
- `apps/hiring/admin.py:117-118`, `apps/common/newsroom/navigation.py:238-239`,
  `apps/accounts/admin_roles.py:56-66`
```

Impacto

Exposição de dados pessoais de candidatos (currículos) a um papel que a interface declara sem acesso.

Sugestão de correção

Reusar a mesma regra do admin, para as duas nunca divergirem:

```
``python
from django.contrib import admin
from django.core.exceptions import PermissionDenied

@staff_member_required
def download_resume(request, application_id):
    if not admin.site.get_model_admin(Application).has_view_permission(request):
        raise PermissionDenied
    application = get_object_or_404(Application, pk=application_id)
```

Alternativa complementar: retirar `hiring` de `GENERAL\_ADMIN\_APP\_LABELS`.

Critérios de aceite

```
- [ ] Administrador Geral não superusuário recebe 403 em `/hiring/application/<id>/resume/`.
- [ ] Superusuário continua baixando o currículo.
- [ ] Teste de regressão em `apps/hiring/tests.py`.
```

--- FIM ISSUE 4 ---

Issue 5 — IDOR-02

--- ISSUE 5 ---

Título: [Segurança] Prévia de newsletter mostra qualquer notícia (inclusive rascunho) a qualquer staff
Labels sugeridas: security, severidade: média

Descrição

`newsletter\_preview` exige só `is\_staff` e busca a notícia por ID com `Article.objects`: não confere `news.view\_article`, status nem Site. O Administrador Komuniki (staff, sem permissão de notícias) lê qualquer notícia; Editor e Repórter (não staff) não alcançam a rota. A rota não é linkada em nenhuma tela.

Por que é explorável

`GET /news/newsletter/preview/<n>/` percorrendo IDs sequenciais devolve o e-mail renderizado de rascunhos e agendadas.

Confirmado com teste dinâmico: Administrador Komuniki sem `news.view\_article` recebeu 200 com o título do rascunho.

Evidência

```
- `apps/news/views.py:524-536`  
```python  
@staff_member_required
def newsletter_preview(request, article_id):
 ...
 article = get_object_or_404(Article, id=article_id)
 ...
```

### ## Impacto

Vazamento de conteúdo não publicado para papéis sem relação com o portal de notícias.

### ## Sugestão de correção

```
```python  
@login_required  
def newsletter_preview(request, article_id):  
    if not request.user.has_perm('news.view_article'):  
        raise PermissionDenied  
    ...  
    article = get_object_or_404(Article.on_site, id=article_id)  
    ...
```

Ou remover a rota, se não houver uso.

Critérios de aceite

- [] Conta staff sem `news.view\_article` recebe 403.
- [] Quem tem `news.view\_article` acessa (ou a rota foi removida e as URLs antigas dão 404).
- [] Busca restrita a `Article.on\_site`; teste de regressão.

--- FIM ISSUE 5 ---

Issue 6 — SEC-01 + SEC-02 + SEC-03 + SEC-04

--- ISSUE 6 ---

Título: [Segurança] Placeholders de segredos passam no deploy (+ defaults de dev e IP de origem)
Labels sugeridas: security, severidade: média

Descrição

`config/settings/base.py:12` exige `SECRET\_KEY`, mas nenhum check rejeita os valores de exemplo publicados no repositório (`env.prod.example:11, 25, 28, 39`; `env.example:2, 10`). O Django só emite o aviso `security.W009` para chave fraca, e o deploy roda `check --deploy --fail-level ERROR` (`scripts/deploy/kellysys-deploy:137`), que deixa avisos passarem.

Itens agrupados, do mesmo tema:

- (baixa) `docker/docker-compose.yml:39-41, 52-53` publica Postgres (senha `news\_portal\_pass`) e Mailpit em `0.0.0.0`.
- (informativa) IP de origem da VPS em `env.prod.example:14` e `docs/technical/cloudflare-bots.md:56, 86, 103`.
- (informativa) Histórico: `base.py` teve `default='django-insecure-change-me-in-production'` entre os commits `a3b8544` e `5033972` (23 a 25/02/2026). A varredura dos 182 commits não achou nenhum segredo real.

Por que é explorável

Com o `env.prod` copiado do exemplo, a `SECRET\_KEY` é pública: dá para forjar tokens de descadastro da newsletter (`apps/news/newsletter.py:43-61`) e cookies de mensagens, e o pepper HMAC dos códigos de verificação deixa de ser segredo. O banco de dev e a caixa do Mailpit (com códigos de verificação) ficam acessíveis na rede local.

```

Confirmado: `manage.py check --deploy --fail-level ERROR` com
`SECRET_KEY=__TROQUE_POR_UMA_CHAVE_FORTE__` terminou com exit 0.

## Evidência
```
.env.prod.example:11 SECRET_KEY=__TROQUE_POR_UMA_CHAVE_FORTE__
scripts/deploy/kellysys-deploy:137 manage.py check --deploy --fail-level ERROR
docker/docker-compose.yml:41 - "5432:5432"
```

## Impacto
Segurança da produção dependente só da disciplina do operador; exposição de ambiente de dev;
facilidade para contornar o Cloudflare se o firewall da origem não estiver aplicado.

## Sugestão de correção
```python
apps/common/checks.py (registrar em CommonConfig.ready)
from django.conf import settings
from django.core.checks import Error, Tags, register

PLACEHOLDERS = ('__TROQUE__', '__SENHA__', 'your-secret-key-here', 'django-insecure', 'change-me')

@register(Tags.security, deploy=True)
def check_secret_placeholders(app_configs, **kwargs):
 errors = []
 key = settings.SECRET_KEY
 if len(key) < 50 or len(set(key)) < 5 or any(p in key for p in PLACEHOLDERS):
 errors.append(Error('SECRET_KEY fraca ou de exemplo.', id='common.E010'))
 for name, value in (('DB_PASSWORD', settings.DATABASES['default'].get('PASSWORD', '')),
 ('EMAIL_HOST_PASSWORD', settings.EMAIL_HOST_PASSWORD)):
 if any(p in (value or '') for p in PLACEHOLDERS):
 errors.append(Error(f'{name} ainda com valor de exemplo.', id='common.E011'))
 return errors
```

- Compose de dev: `"127.0.0.1:5432:5432"`, `"127.0.0.1:8025:8025"`, `"127.0.0.1:1025:1025"`.
- Trocar o IP por `<IP_DA_VPS>` no exemplo e nos docs; confirmar `curl -I --max-time 5 http://<IP>/`
  → timeout.
- Secret scanning no CI (ex.: gitleaks); rotacionar a `SECRET_KEY` se algum ambiente subiu entre 23
  e 25/02/2026.

## Critérios de aceite
- [ ] `manage.py check --deploy --fail-level ERROR` falha com a `SECRET_KEY` ou senhas de exemplo
  (teste do check).
- [ ] 0 compose de dev publica portas só em `127.0.0.1`.
- [ ] 0 IP de origem não aparece mais no repositório e o firewall da origem foi confirmado.
- [ ] Job de secret scanning no CI; `SECRET_KEY` rotacionada, se aplicável.

```

--- FIM ISSUE 6 ---

Issue 7 — XSS-02 + XSS-03

--- ISSUE 7 ---

```

Título: [Segurança] Endurecer a CSP de script-src e remover o |safe residual
Labels sugeridas: security, severidade: baixa

## Descrição
A CSP do Django (`config/settings/base.py:454-459`) e a do nginx (`docker/nginx/nginx.conf:167`)
permitem `unsafe-inline` e `unsafe-eval` em `script-src`, o que anula a CSP como mitigação de
XSS (é o que torna o XSS de upload executável). Há também um `|safe` residual em
`templates/accounts/password_reset/new_password.html:44`, contra a regra do projeto – hoje
inofensivo (o help text vem dos validadores do Django).

## Por que é explorável
Não é explorável sozinho: amplifica qualquer injeção que escape do autoescape ou da sanitização.

## Evidência
```python
config/settings/base.py:454-459 (resumido em uma linha)
'script-src': [SELF, UNSAFE_INLINE, UNSAFE_EVAL, 'https://challenges.cloudflare.com'],
```
```django
{% templates/accounts/password_reset/new_password.html:44 %}
<div class="mt-1 text-xs text-slate-500 font-ui">{{ field.help_text|safe }}</div>
```

## Impacto

```

Defesa em profundidade ausente contra XSS.

Sugestão de correção

- Trocar o Alpine pelo build CSP (`@alpinejs/csp`) para remover ``unsafe-eval``.
- Mover scripts inline para `static/` ou usar nonce do django-csp (`{{ request.csp\_nonce }}`) e retirar ``unsafe-inline`` de `script-src`, mantendo Django e nginx com políticas iguais (o navegador aplica a interseção).
- Trocar `{{ field.help\_text|safe }}` por `{{ field.help\_text|escape|safe }}` (o texto do Django já vem marcado como seguro).

Critérios de aceite

- [] Nenhum `|safe` em `templates/` (verificação no CI).
- [] `script-src` sem ``unsafe-eval`` e sem ``unsafe-inline`` no Django e no nginx.
- [] Portais e painéis sem violações de CSP no console (testes E2E).

--- FIM ISSUE 7 ---